

Feature Completion Report

Project & Team Task Management Platform

This report provides a comprehensive evaluation of the platform against the requested Core Features and Technical Requirements.

1. Core Features Evaluation

☒ Administrator

Requirement: Manage users, roles, projects, and overall system access.

Status: COMPLETED

Implementation Details:

- Dedicated `ADMIN` role enforced via JWT middleware.
- Full CRUD access to `/api/users/` (Create, Read, Update, Delete users and assign roles).
- Unrestricted access to view, edit, and delete all system projects and tasks.
- Exclusive access to `/api/activity/` to view system-wide activity logs.

☒ Project Manager

Requirement: Create and manage projects, assign team members, and manage project-related tasks.

Status: COMPLETED

Implementation Details:

- `PROJECT_MANAGER` role implemented.
- Can create new projects via `POST /api/projects/`.
- Can manage their own projects (Update/Delete).
- Can assign members to their projects via `POST /api/projects/:id/members`.
- Can create, assign, and manage tasks within their projects via `/api/tasks/`.

☒ Team Member

Requirement: View assigned projects and tasks, update task progress, and perform permitted task-related activities.

Status: COMPLETED

Implementation Details:

- `TEAM_MEMBER` role is the default assignment.
- Can only view projects they have been explicitly added to (via `project_members` join table).
- Can only view tasks assigned to them.
- Permitted to update their task progress (`TODO`, `IN_PROGRESS`, `IN_REVIEW`, `DONE`) via `PATCH /api/tasks/:id/progress`.
- Can leave comments on tasks via `POST /api/tasks/:id/comments`.

2. Technical Requirements Evaluation

☒ Frontend

Requirement: Next.js

Status: COMPLETED

Implementation Details: The frontend application is built using Next.js, leveraging React and Tailwind CSS for rapid, responsive UI development.

☑ Backend

Requirement: Laravel or Node.js

Status: COMPLETED

Implementation Details: The backend API is entirely constructed using Node.js and Express.js, providing a robust JavaScript ecosystem from front to back.

☑ Database

Requirement: MySQL or PostgreSQL

Status: COMPLETED

Implementation Details: The platform uses a cloud-hosted MySQL database (hosted on Aiven), utilizing the `mysql2` package for connection pooling and efficient querying.

☑ Secure authentication and role-based access control

Requirement: Secure auth and RBAC.

Status: COMPLETED

Implementation Details:

- Users are authenticated using `bcrypt` for password hashing and JSON Web Tokens (JWT) for stateless session management.
- An `authorize("ROLE")` middleware strictly enforces access control on backend routes.

☑ RESTful API integration

Requirement: RESTful API.

Status: COMPLETED

Implementation Details: The frontend seamlessly communicates with the Node.js backend using the Axios HTTP client, targeting fully RESTful endpoints (GET, POST, PUT, PATCH, DELETE).

☑ Proper database relationships and validation

Requirement: Relational integrity.

Status: COMPLETED

Implementation Details: The MySQL schema heavily relies on Primary Keys and Foreign Keys with `ON DELETE CASCADE` and `ON DELETE SET NULL` constraints to ensure orphan data is never left behind (e.g., deleting a project deletes all its tasks and members).

☑ Responsive and user-friendly interface

Requirement: Responsive UI.

Status: COMPLETED

Implementation Details: The Next.js frontend uses Tailwind CSS, employing a mobile-first design approach to ensure the application is usable across all device sizes (Desktops, Tablets, Mobile).

☑ Git-based version control

Requirement: Version Control.

Status: **COMPLETED**

Implementation Details: The entire monorepo is continuously tracked and synced to a centralized GitHub repository.

☒ **Basic CI/CD pipeline for linting, testing, or build validation**

Requirement: CI/CD pipeline.

Status: **COMPLETED**

Implementation Details:

- The application is deployed via Vercel.
- Vercel automatically acts as a CI/CD pipeline: every push to the GitHub `main` branch triggers an automatic build validation and seamless deployment of both the Next.js frontend and Node.js Serverless Functions.

Conclusion:

All foundational requirements and core features have been **100% completed**, successfully integrated, and deployed to production.